

Logsum Exp Trick

q_{1,k_1}	q_{1,k_2}
q_{2,k_1}	q_{2,k_2}

Take $\vec{x}_k = (x_i)_{i=1}^n$

$$\text{Softmax}(x_i) = \frac{\exp(x_i - x_{\max})}{\left(\sum_{j=1}^N \exp(x_j - x_{\max})\right)} \rightarrow l_k$$

$\nearrow m_k$

Will save in forward: $m_k + \log(l_k) =: \underline{}$ (one qty instead of two)

$$\begin{aligned} \text{Softmax}^2(x_i) &= \exp(x_i - m_k - \log(l_k)) \\ &= \frac{\exp(x_i - m_k)}{\exp(\log(l_k))} = \frac{\exp(x_i - m_k)}{l_k} \end{aligned}$$

Review of Gradients and Jacobians

$$f: \mathbb{R}^N \rightarrow \mathbb{R}$$

$$f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = y$$

$$\frac{\partial y}{\partial \vec{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \end{bmatrix} \rightarrow \text{Gradient}$$

$$f: \mathbb{R}^N \rightarrow \mathbb{R}^M$$

$$f\left(\begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix}\right) = \begin{bmatrix} y_1 \\ \vdots \\ y_M \end{bmatrix}$$

$$\frac{\partial \vec{y}}{\partial \vec{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_N} \\ \vdots & & \vdots \\ \frac{\partial y_M}{\partial x_1} & \cdots & \frac{\partial y_M}{\partial x_N} \end{bmatrix} \rightarrow \text{Numerator layout}$$

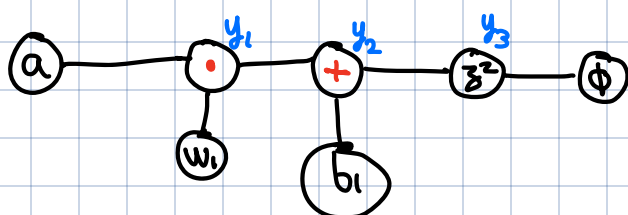
Generalized Jacobians with Tensors:

$$f: \mathbb{R}^{N_1 \times \dots \times N_{D_x}} \rightarrow \mathbb{R}^{M_1 \times \dots \times M_{D_y}}$$

↳ D_x -dimensional input

$$\frac{\partial \vec{y}}{\partial \vec{x}} : (M_1 \times \dots \times M_{D_y}) \times (N_1 \times \dots \times N_{D_x})$$

Autograd



$$\phi = (a \cdot w_1 + b_1)^2 = y_1 + b_1 = y_2 = y_3$$

$$\frac{\partial \phi}{\partial w_1} = 2a(a w_1 + b_1)$$

$$\frac{\partial \phi}{\partial w_1} = \frac{\partial \phi}{\partial y_3} \cdot \frac{\partial y_3}{\partial y_2} \cdot \frac{\partial y_2}{\partial y_1} \cdot \frac{\partial y_1}{\partial w_1}$$

PyTorch: Each function has Forward + Backward
 - It goes to each y_i , and, given the accumulated gradient $(\frac{\partial \phi}{\partial y_i} = \frac{\partial \phi}{\partial y_n} \dots \frac{\partial y_{i+1}}{\partial y_i})$, it asks node y_i

to calculate gradients w.r.t. inputs.

$$y_i = y_{i-1} W \quad \xrightarrow{[N, D] \rightarrow [D, M]} \quad \rightarrow N, M \text{ are large, so } y_i \text{ will be huge}$$

← $[N, M]$

$$\frac{\partial \phi}{\partial y_{i-1}} = \frac{\partial \phi}{\partial y_i} \cdot \frac{\partial y_i}{\partial y_{i-1}} \quad \xrightarrow{\text{Upstream Gradient}} \quad \text{Local Jacobian}$$

$(N \cdot M) \times (N \cdot D) \rightarrow$ Too big to store

↑ Downstream Gradient

(We don't want to materialize the Jacobian fully)

But, $\frac{\partial y_i}{\partial y_{i-1}}$ is a very sparse matrix:

$$y_i = y_{i-1} W = \begin{bmatrix} \dots \end{bmatrix}_{(N,D)} \begin{bmatrix} \vdots \end{bmatrix}_{(D,M)} = \begin{bmatrix} \dots \end{bmatrix}_{(N,M)}$$

Row k : bunch of dot products with row $y_{i-1,k}$ and W . So grads will be mostly zero
 Thus, $\frac{\partial y_i}{\partial y_{i-1}(mn)} = 0$ unless $m=k$

Gradient of Matmul:

$$y = XW$$

PyTorch will give us the upstream gradient: $\frac{\partial \phi}{\partial y}$

We must calculate $\frac{\partial \phi}{\partial X}$, $\frac{\partial \phi}{\partial W}$

$$\text{Let } X: [N, D], W: [D, M], Y: [N, M]$$

Consider $\frac{\partial \phi}{\partial X}$ first:

$$Y = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ \vdots & \vdots & & \vdots \\ x_{N1} & \dots & \dots & x_{ND} \end{bmatrix} \begin{bmatrix} w_{11} & \dots & w_{1M} \\ \vdots & & \vdots \\ w_{D1} & \dots & w_{DM} \end{bmatrix}$$

$$= \begin{bmatrix} \sum_{i=1}^D x_{1i} w_{i1} & \sum_{i=1}^D x_{1i} w_{i2} & \dots & \sum_{i=1}^D x_{1i} w_{iM} \\ \vdots & \vdots & & \vdots \\ \sum_{i=1}^D x_{Ni} w_{i1} & \dots & \dots & \sum_{i=1}^D x_{Ni} w_{iD} \end{bmatrix}$$

$$\frac{\partial \phi}{\partial y} = \begin{bmatrix} \frac{\partial \phi}{\partial y_{11}} & \dots & \frac{\partial \phi}{\partial y_{1M}} \\ \vdots & & \vdots \\ \frac{\partial \phi}{\partial y_{N1}} & \dots & \frac{\partial \phi}{\partial y_{NM}} \end{bmatrix} \rightarrow \text{Given}$$

$$\frac{\partial \phi}{\partial x} = \frac{\partial \phi}{\partial y} \cdot \frac{\partial y}{\partial x}$$

↳ Local gradient / Jacobian

Step 1: Vectorize y and $x \rightarrow$ Row-major

$$\text{Vec}(y) = \begin{bmatrix} y_{11} \\ y_{12} \\ \vdots \\ y_{1M} \\ y_{21} \\ \vdots \\ y_{2M} \\ \vdots \\ y_{NM} \end{bmatrix}$$

$$\text{Vec}(x) = \begin{bmatrix} x_{11} \\ x_{12} \\ \vdots \\ x_{1D} \\ x_{21} \\ \vdots \\ x_{2D} \\ \vdots \\ x_{N1} \\ \vdots \\ x_{ND} \end{bmatrix}$$

$$J_x = \frac{\partial \text{Vec}(y)}{\partial \text{Vec}(x)} = \begin{bmatrix} \frac{\partial y_{11}}{\partial x_{11}} & \dots & \frac{\partial y_{11}}{\partial x_{ND}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_{NM}}{\partial x_{11}} & \dots & \frac{\partial y_{NM}}{\partial x_{ND}} \end{bmatrix}$$

(NM, ND)

Step 2: Compute each entry in Jacobian

$$\frac{\partial y_{ij}}{\partial x_{kl}} = \frac{\partial}{\partial x_{kl}} \left(\sum_{s=1}^D x_{ks} w_{sj} \right) = \begin{cases} 0 & \text{if } k \neq i \\ w_{ij} & \text{if } k = i \end{cases}$$

$$J_x = \begin{bmatrix} W^T & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & W^T \end{bmatrix} = \mathbb{I} \otimes W^T \quad (\text{Tensor product})$$

Step 3: Simplify

Notice that

$$\text{Vec} \left(\frac{\partial \phi}{\partial x} \right) = J_x^T \text{Vec} \left(\frac{\partial \phi}{\partial y} \right) \quad (\text{Can be derived})$$

↳ By calculating this and then reshaping it,
we obtain $\frac{\partial \phi}{\partial x} = \frac{\partial \phi}{\partial y} W^T$ (VSP)

[N, D] [N, M] [M, D]

Similarly, $\frac{\partial \phi}{\partial \mathbf{W}} = \mathbf{X}^T \frac{\partial \phi}{\partial \mathbf{y}}$ (VSP)

$\begin{matrix} \frac{\partial \phi}{\partial \mathbf{W}} & \frac{\partial \phi}{\partial \mathbf{y}} \\ [D, M] & [D, N] \quad [N, N] \end{matrix}$

In practise, with a scalar loss L , we compute

$$\left(\frac{\partial L}{\partial \mathbf{X}} \right)_{ab} = \sum_{i=1} \sum_{j=1} \frac{\partial L}{\partial y_{ij}} \frac{\partial y_{ij}}{\partial x_{ab}}$$

Gradient of Softmax:

$$S = QK^T \quad P = \text{Softmax}(S)$$

$$S = \begin{bmatrix} s_1 \\ \vdots \\ s_n \end{bmatrix}$$

$$P = \begin{bmatrix} p_1 \\ \vdots \\ p_n \end{bmatrix} = \begin{bmatrix} \text{Softmax}(s_1) \\ \vdots \\ \text{Softmax}(s_n) \end{bmatrix}$$

$$\frac{\partial \phi}{\partial S} = \frac{\partial \phi}{\partial P} \cdot \frac{\partial P}{\partial S}$$

$$\frac{\partial P}{\partial S} = \begin{bmatrix} \frac{\partial p_1}{\partial s_1} & \frac{\partial p_1}{\partial s_2} & \dots & \frac{\partial p_1}{\partial s_n} \\ \vdots & \vdots & & \vdots \\ \frac{\partial p_n}{\partial s_1} & \dots & \dots & \frac{\partial p_n}{\partial s_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial p_1}{\partial s_1} & \dots & 0 \\ \vdots & & \vdots \\ 0 & \dots & \frac{\partial p_n}{\partial s_n} \end{bmatrix}$$

$$\frac{\partial p_i}{\partial s_i} = \frac{\partial}{\partial s_i} \left[\frac{e^{x_{ik}}}{\sum_{j=1}^n e^{x_{ij}}} \right]_{1 \leq k \leq n} = \begin{bmatrix} \frac{\partial p_{i1}}{\partial s_{i1}} & \frac{\partial p_{i1}}{\partial s_{i2}} & \dots & \frac{\partial p_{i1}}{\partial s_{in}} \\ \vdots & \vdots & & \vdots \\ \frac{\partial p_{in}}{\partial s_{i1}} & \frac{\partial p_{in}}{\partial s_{i2}} & \dots & \frac{\partial p_{in}}{\partial s_{in}} \end{bmatrix}$$

$$s_i = [s_{i1} \dots s_{in}]$$

$$\frac{\partial p_{ik}}{\partial s_{il}} : \quad k=n \quad l=1 \quad n=k$$

$$\text{Case 1: } k=l \quad \frac{\partial p_{ik}}{\partial s_{ik}} = \frac{\partial}{\partial s_{ik}} \left(\frac{e^{x_{ik}}}{\sum_{j=1}^n e^{x_{ij}}} \right) = \frac{e^{x_{ik}} \sum_{j=1}^n e^{x_{ij}} - e^{x_{ik}} e^{x_{ik}}}{\left(\sum_{j=1}^n e^{x_{ij}} \right)^2}$$

$$\text{Let } s_{ik} = \frac{e^{x_{ik}}}{\sum_{j=1}^n e^{x_{ij}}}$$

$$= \frac{e^{x_{ik}} \left(\sum_{j=1}^n e^{x_{ij}} - e^{x_{ik}} \right)}{\left(\sum_{j=1}^n e^{x_{ij}} \right)^2} = s_{ik}(1-s_{ik})$$

$$\text{Case 2: } k \neq l \quad \frac{\partial P_{ik}}{\partial S_{il}} = \frac{\partial}{\partial S_{il}} \left(\frac{e^{x_{ik}}}{\sum_{j=1}^n e^{x_{ij}}} \right) = \frac{0 - e^{x_{ik}} e^{x_{il}}}{\left(\sum_{j=1}^n e^{x_{ij}} \right)^2} = -S_{ik} S_{il} = -S_{il} S_{ik}$$

$$\text{So } \frac{\partial P_i}{\partial S_i} = \begin{bmatrix} S_{i1} & 0 \\ 0 & S_{in} \end{bmatrix} + \begin{bmatrix} -S_{i1}S_{i1} & \dots & -S_{i1}S_{in} \\ \vdots & \ddots & \vdots \\ S_{in}S_{i1} & \dots & -S_{in}S_{in} \end{bmatrix}$$

$$= \text{diag}(S_i) - S_i S_i^T$$

Derivative wrt each row

$$\text{For our kernel, we will need to take } \frac{\partial \phi}{\partial S_i} = \frac{\partial \phi}{\partial P_i} \cdot \frac{\partial P_i}{\partial S_i} \xrightarrow{= \text{vec}^n}$$

$$= V^T (\text{diag}(S_i) - S_i S_i^T)$$

Attention Backward Pass (Classical): Need dQ, dk, dV

$$O = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V = PV$$

$$\frac{\partial \phi}{\partial P} = \frac{\partial \phi}{\partial O} \cdot \frac{\partial O}{\partial P} = \frac{\partial \phi}{\partial O} \cdot V^T = dO \cdot V^T$$

$$\frac{\partial \phi}{\partial V} = P^T \frac{\partial \phi}{\partial O} = P^T dO$$

$$O = PV$$

$$\text{Row } i \rightarrow O_i = \sum_{j=1}^n P_{ij} V_j$$

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ b_{n1} & \dots & b_{nn} \end{bmatrix}$$

$$AB_1 = a_{11} B_1 + a_{12} B_2 + \dots + a_{1n} B_n$$

$$\frac{\partial \phi}{\partial w} \rightarrow \frac{\partial V_j}{\partial w} = \sum_{i=1}^n P_{ij} \frac{\partial O_i}{\partial w} = \sum_{i=1}^n \frac{e^{a_{ij}}}{Z_i} dO_i$$

$\uparrow$
 $P^T dO$ $\uparrow$
 $\frac{\partial \phi}{\partial w}$

$$P_i = \text{Softmax}(S_i)$$

$$\partial S_i = (\text{diag}(P_i) - P_i P_i^T) \partial P_i = P_i \circ \partial P_i - \overbrace{(P_i^T \partial P_i)}^{\text{Scalar, so commutative}} P_i$$

(We usually write $\frac{\partial \phi}{\partial S_i} = \frac{\partial \phi}{\partial P_i} \cdot \frac{\partial P_i}{\partial S_i}$, where $\frac{\partial P_i}{\partial S_i}$ is in numerator form. Here, we are writing it in denominator form, and $(\frac{\partial P_i}{\partial S_i})_{\text{num}} = (\frac{\partial P_i}{\partial S_i})_{\text{den}}^T$. So we have $(\frac{\partial \phi}{\partial S_i})^T = (\frac{\partial P_i}{\partial S_i})^T \cdot (\frac{\partial \phi}{\partial P_i})^T$)

Column vectors

$$\text{Given } S_{ij} = q_i^T k_j$$

$$\partial q_i = \frac{\partial \mathcal{L}}{\partial q_i} = \sum_j \frac{\partial \mathcal{L}}{\partial S_{ij}} \cdot \frac{\partial S_{ij}}{\partial q_i} = \sum_j \partial S_{ij} \cdot k_j$$

$$\partial k_j = \sum_i \partial S_{ij} q_i$$

Define

$$D_i = P_i^T \cdot dP_i = \sum_j \frac{e^{q_i^T k_j}}{L_i} d o_i^T v_j = d o_i^T \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j = d o_i^T o_i,$$

then

$$dS_i = P_i \circ dP_i - D_i P_i.$$

Hence

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij} (dP_{ij} - D_i).$$

Now we can get the gradients dQ and dK . Recall that $S_{ij} = q_i^T k_j$, so

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij} (dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} (d o_i^T v_j - D_i) k_j.$$

→ Precompute

Similarly,

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij} (dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} (d o_i^T v_j - D_i) q_i.$$

Algorithm 4 FLASHATTENTION Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} , pseudo-random number generator state $\mathcal{R}$ from the forward pass.

- 1: Set the pseudo-random number generator state to $\mathcal{R}$.
 - 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ into T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: Initialize $\tilde{\mathbf{dK}}_j = (0)_{B_c \times d}, \tilde{\mathbf{dV}}_j = (0)_{B_c \times d}$ on SRAM.
 - 9: **for** $1 \leq i \leq T_r$ **do**
 - 10: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 11: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 12: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
 - 13: On chip, compute $\mathbf{P}_{ij} = \text{diag}(\ell_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
 - 14: On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop} .
 - 15: On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
 - 16: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_{ij}^{\text{dropped}})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
 - 17: On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 18: On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
 - 19: On chip, compute $\mathbf{D}_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
 - 20: On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - \mathbf{D}_i) \in \mathbb{R}^{B_r \times B_c}$.
 - 21: Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
 - 22: On chip, compute $\tilde{\mathbf{dK}}_j \leftarrow \tilde{\mathbf{dK}}_j + \tau \mathbf{dS}_{ij}^T \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
 - 23: **end for**
 - 24: Write $\mathbf{dK}_j \leftarrow \tilde{\mathbf{dK}}_j, \mathbf{dV}_j \leftarrow \tilde{\mathbf{dV}}_j$ to HBM.
 - 25: **end for**
 - 26: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-